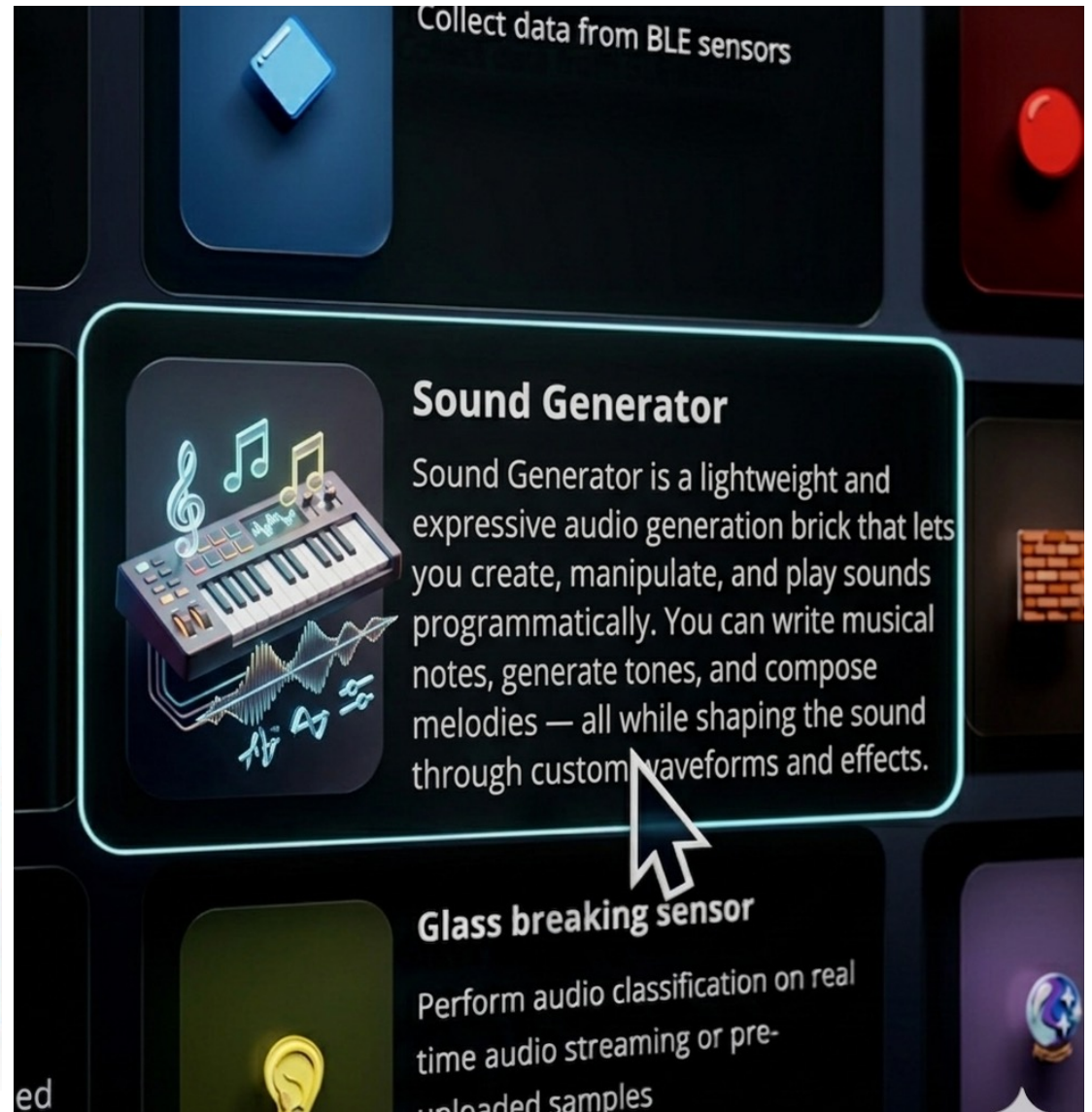
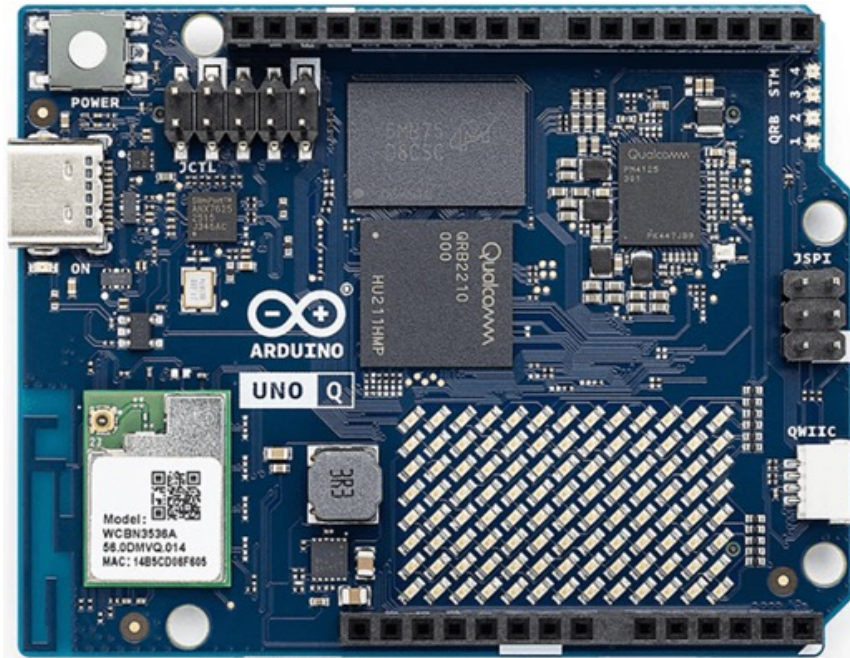




บล็อก (Bricks)



Sound Generator

Generate sounds like notes, tones, or melodies using waveforms.

🎵 Sound Generator Brick: สร้างเสียงได้ง่าย ๆ บน Arduino App Lab!

อีกหนึ่ง Brick ใหม่สุดเจ๋งใน Arduino App Lab ที่สายทำโปรเจกต์ต้องลองก็คือ **Sound Generator** 🎵 ช่วยให้เรา “สร้างเสียงจากโค้ด” ได้แบบมือโปร โดยไม่ต้องมีความรู้ด้านเสียงลึกมาก!

🦉 Sound Generator คืออะไร?

เป็น Brick สำหรับสร้างและควบคุมเสียงแบบ programmatic

👉 ทำได้ทั้ง:

- สร้างโน้ตดนตรี 🎵
- สร้างเสียง tone ต่าง ๆ
- แต่ง melody ได้เองจากโค้ด

ใส่เอฟเฟกต์เสียงได้ เช่น:

- chorus
- delay
- vibrato
- distortion / overdrive

เล่นเสียงแบบ **real-time** ผ่านลำโพง 🗣️

💡 ฟิเจอร์เด่น

สร้างเสียงจาก **โน้ต** หรือ **ความถี่ (frequency)** เลือก waveform ได้:

- sine
- square
- triangle
- sawtooth

💡 เอาไปทำอะไรได้บ้าง?

- 🔔 เสียงแจ้งเตือนอัจฉริยะ
- 🎮 เกมบน Arduino UNO Q
- 🎹 สร้างเครื่องดนตรี DIY
- 🤖 ให้ AI ตอบสนองด้วยเสียง
- 🎵 สร้าง procedural music จากโค้ด

🔥 จุดเด่นจริง ๆ

ไม่ใช่แค่ “มีเสียง” แต่คือ

👉 เราสามารถ “ออกแบบเสียงเอง” ได้เลย
= จากบอร์ดยึด → กลายเป็น Audio Device ได้ทันที

📌 สรุป

Sound Generator Brick =
“เปลี่ยนโค้ดให้กลายเป็นเสียงได้แบบ real-time พร้อมเอฟเฟกต์ระดับโปร”

ทำไมต้องมี "sine" / "square" / "sawtooth" / "triangle" แค่ 4 รูปแบบนี้?

เพราะ 4 waveform นี้คือพื้นฐานของเสียงทุกชนิด และ waveform อื่น ๆ แทบทั้งหมด “สร้างมาจากมันได้”

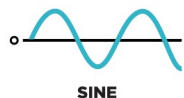
Sine — เสียงบริสุทธิ์ที่สุด

- มีความถี่เดียว
- ไม่มี harmonic อื่น
- เสียงนุ่ม สะอาด

👉 ใช้เป็น:

- sub bass
- tone ทดสอบ
- เสียง reference

sine = อะตอมของเสียง



Square — เสียงหนา ดิบ

- มี odd harmonics
- เสียง buzz แต่ยังควบคุมได้

👉 ใช้เป็น:

- synth ดุดัน
- lead คม
- เสียง digital / retro

square = sine หลายตัวรวมกันแบบเฉพาะ



Triangle — ทางสายกลาง

- มี odd harmonics เหมือน square
- แต่ความแรงของ harmonic ลดลงเรื่อย ๆ

👉 ใช้เป็น:

- lead ใส
- เสียงคล้ายเครื่องเป่า
- ไม่แข็ง ไม่บาง

triangle = square ที่ “สุภาพขึ้น”



Sawtooth — harmonic แน่นที่สุด

- มี harmonic ทุกลำดับ
- เสียงสว่าง แรง buzz สูง

👉 ใช้เป็น:

- pad
- supersaw
- subtractive synthesis

saw = วัตถุดิบชั้นดีสำหรับ “เอาไปกรอง”



4 ตัวนี้ = ครบคลุมเสียงพื้นฐานทั้งหมด

ถ้าคุณควบคุม:

- waveform
- frequency
- amplitude
- envelope (attack/release)
- filter (ถ้ามี)

คุณสร้างเสียงได้แทบทุกแบบแล้ว

4 waveform นี้ = รากฐานของเสียงทั้งหมด

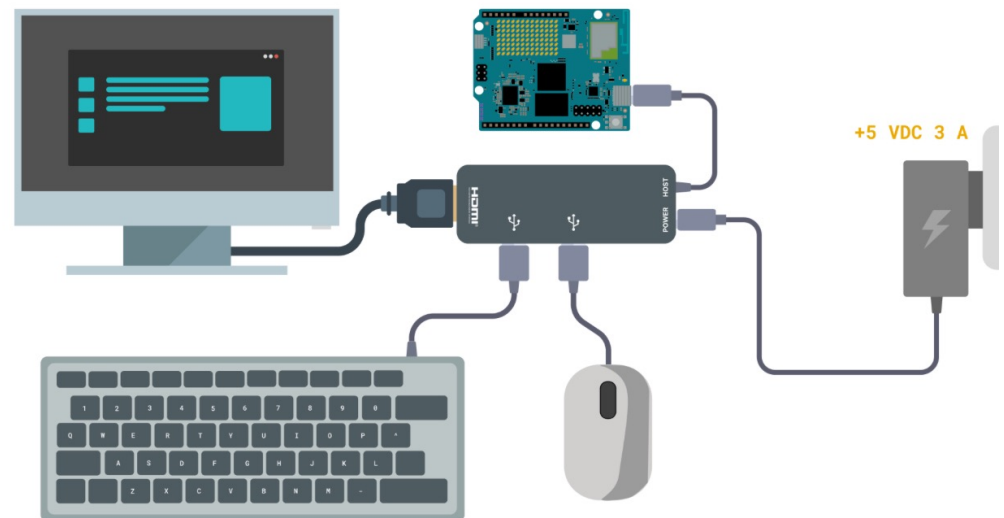
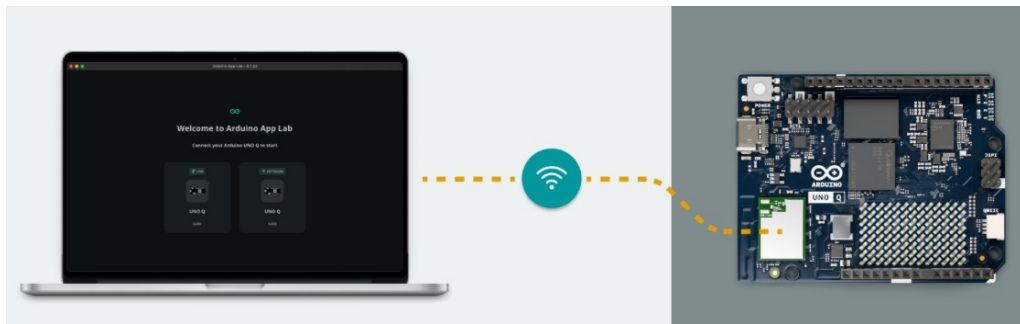
Arduino / Wave Generator ให้มาเพราะ:

- พอ
- ครบ
- ใช้จริง
- เป็นมาตรฐานโลกเสียง

ข้อกำหนดเบื้องต้น (Prerequisites)

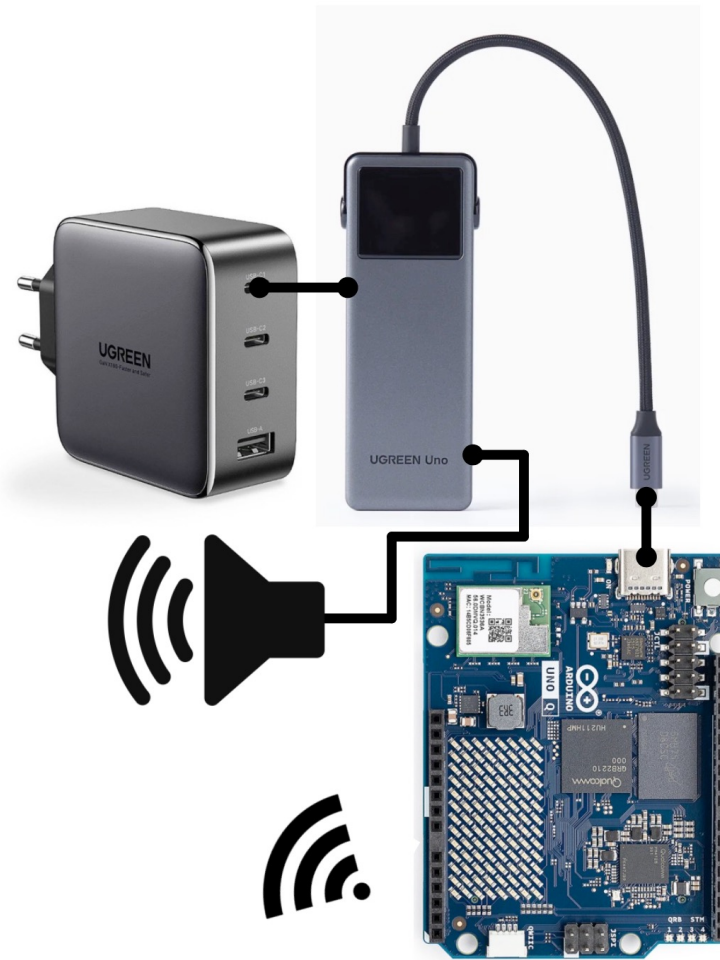
ก่อนใช้งาน Sound Generator brick ต้องมีอุปกรณ์ดังนี้:

- USB-C® Hub พร้อมแหล่งจ่ายไฟภายนอก (5V, 3A)
- อุปกรณ์เสียงแบบ USB (ลำโพง USB หรืออะแดปเตอร์ USB-C → 3.5 มม.)
- Arduino UNO Q ที่ทำงานในโหมด Network Mode หรือ SBC Mode (ต้องใช้พอร์ต USB-C สำหรับฮับ)





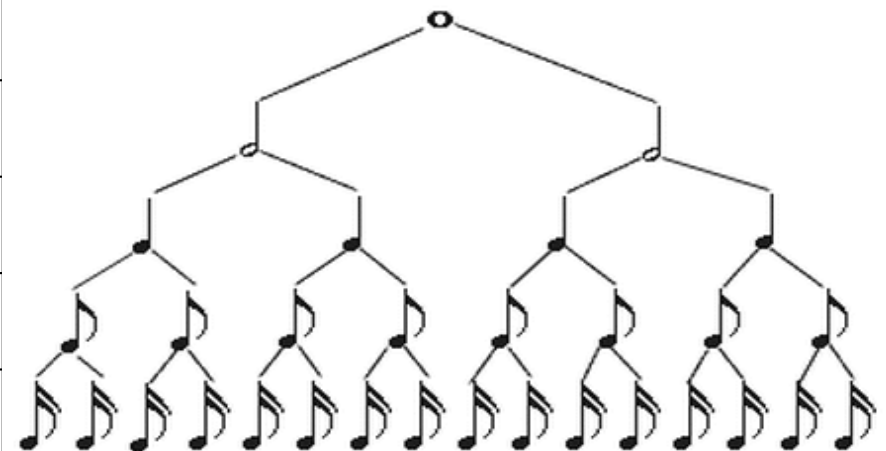
TP-Link_1939

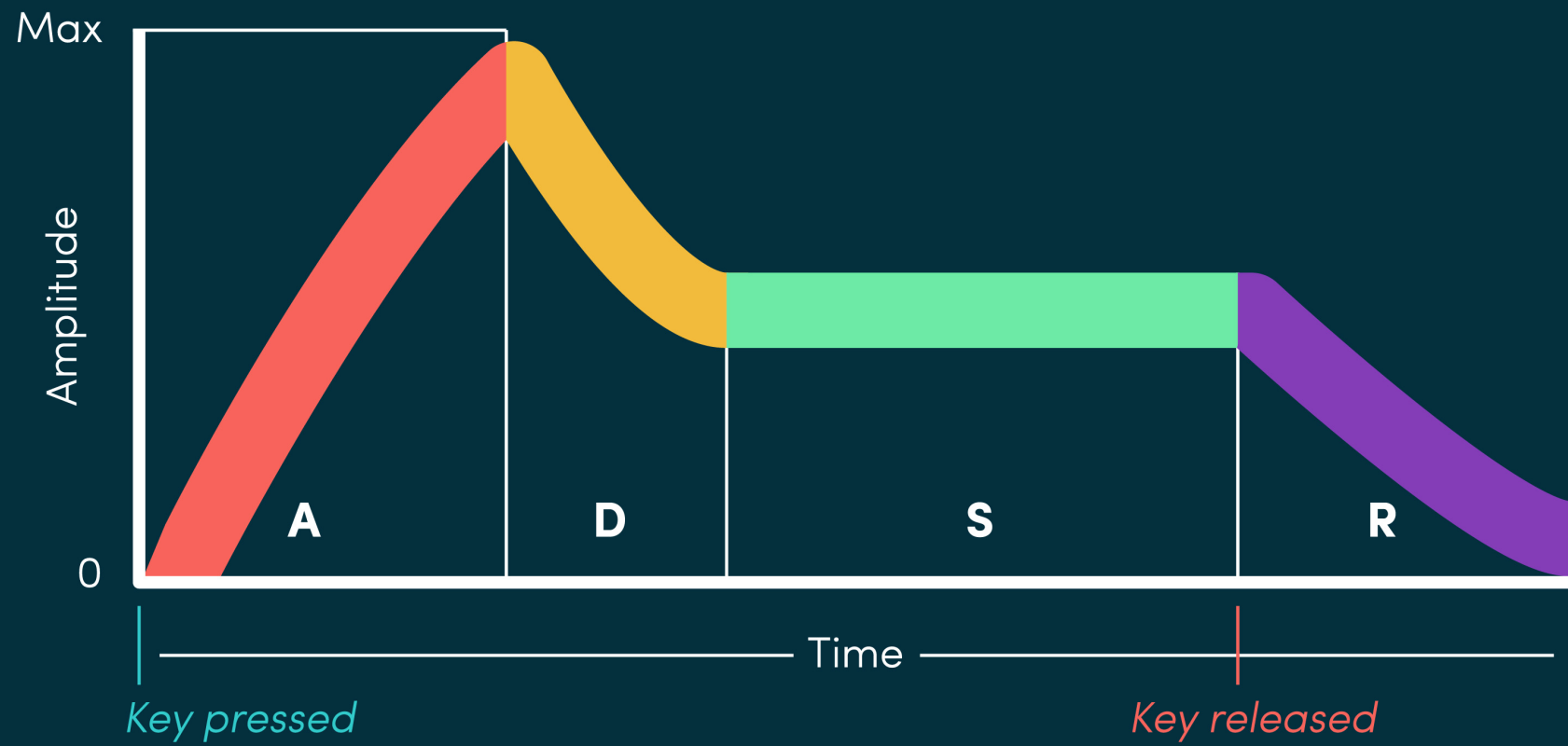


ตำแหน่งของโน้ต

E4	F4	F#4	G4	G#4	A4	A#4	B4	C5	C#5	D5	D#5	E5	F5	F#5	G5	G#5	A5	A#5	B5
B3	C4	C#4	D4	D#4	E4	F4	F#4	G4	G#4	A4	A#4	B4	C5	C#5	D5	D#5	E5	F5	F#5
G3	G#3	A3	A#3	B3	C4	C#4	D4	D#4	E4	F4	F#4	G4	G#4	A4	A#4	B4	C5	C#5	D5
D3	D#3	E3	F3	F#3	G3	G#3	A3	A#3	B3	C4	C#4	D4	D#4	E4	F4	F#4	G4	G#4	A4
A2	A#2	B2	C3	C#3	D3	D#3	E3	F3	F#3	G3	G#3	A3	A#3	B3	C4	C#4	D4	D#4	E4
E2	F2	F#2	G2	G#2	A2	A#2	B2	C3	C#3	D3	D#3	E3	F3	F#3	G3	G#3	A3	A#3	B3

divisor	ประเภทโน้ต	เวลาเล่น
1	whole note (ตัวกลม)	1.0 วินาที
2	half note (ตัวขาว)	0.5 วินาที (1/2)
4	quarter note (ตัวดำ)	0.25 วินาที (1/4)
8	eighth note (เขยี้ต 1 ชั้น)	0.125 วินาที (1/8)
16	sixteenth note (เขยี้ต 2 ชั้น)	0.0625 วินาที (1/16)





ตัวอย่างที่ 1

```
from arduino.app_bricks.sound_generator import SoundGenerator
from arduino.app_utils import App
import time

player = SoundGenerator()

def loop():
    player.play("A4", 1/4)
    time.sleep(1)
    player.play("B4", 1/4)
    time.sleep(1)
    player.play("C5", 1/4)
    time.sleep(1)

App.run(user_loop=loop)
```


ตัวอย่างที่ 2

```
from arduino.app_bricks.sound_generator import SoundGenerator
from arduino.app_utils import App
import time

player = SoundGenerator()

song = [("A4", 1/4), ("B4", 1/4), ("C5", 1/4)]

def loop():
    for note, duration in song:
        player.play(note, duration)
        print(note, duration)

    #player.play("A4", 1/4)
    #time.sleep(1)
    #player.play("B4", 1/4)
    #time.sleep(1)
    #player.play("C5", 1/4)
    #time.sleep(1)

App.run(user_loop=loop)
```

ตัวอย่างค่าพารามิเตอร์ที่ใช้บ่อย (ตามตัวอย่าง)

```
player = SoundGenerator(  
    wave_form="sine",  
    master_volume=0.8,  
    sound_effects=[  
        SoundEffect.adsr(attack=0.05, decay=0.05, sustain=0.7, release=0.05),  
        SoundEffect.overdrive(drive=100.0),  
        SoundEffect.chorus(depth_ms=10, rate_hz=1.0, mix=0.5)  
    ]  
)
```

ตัวอย่างที่ 5

```
from arduino.app_bricks.sound_generator import SoundGenerator
from arduino.app_utils import App
import time

player = SoundGenerator()

def loop():
    player.play_abc("""
X:1
T:Simple Song
M:4/4
K:C
C D E F G
""")

App.run(user_loop=loop)
```

SoundGenerator (คลาสหลัก)

พารามิเตอร์	ความหมาย	ช่วงค่า	ค่า default
wave_form	รูปแบบคลื่น	"sine", "square", "triangle", "sawtooth"	"sine"
master_volume	ระดับเสียงรวม	0.0 – 1.0	0.8
octaves	จำนวนอ็อกเทฟที่เล่นได้	≥ 1 (ปกติ 2+)	2
bpm	ความเร็วเพลง	≥ 1	120 (สำหรับ MusicComposition)
sound_effects	รายการเอฟเฟกต์	list ของ SoundEffect	[SoundEffect.adsr()] สำหรับ MusicComposition

SoundEffectOverdrive

พารามิเตอร์	ความหมาย	ช่วงค่า	ค่า default
drive	ความแรงของ overdrive	≥ 0 (แนะนำ 0–200)	ไม่ระบุ

SoundEffectADSR

พารามิเตอร์	ความหมาย	ช่วงค่า	ค่า default
attack	Attack time (วินาที)	≥ 0	ไม่ระบุ
decay	Decay time (วินาที)	≥ 0	ไม่ระบุ
sustain	Sustain level	0.0 – 1.0	ไม่ระบุ
release	Release time (วินาที)	≥ 0	ไม่ระบุ

SoundEffectChorus

พารามิเตอร์	ความหมาย	ช่วงค่า	ค่า default
depth_ms	ความลึกของ chorus (ms)	0 – 50	10 (จากตัวอย่างของคุณ)
rate_hz	ความเร็ว modulation (Hz)	0.0 – 5.0	1.0
mix	สัดส่วนสัญญาณเอฟเฟกต์กับต้นฉบับ	0.0 – 1.0	0.5

LRUDict class

คลาส LRUDict ใช้สำหรับทำ cache แบบจำกัดขนาด โดยจะลบข้อมูลที่ไม่ได้ใช้งานนานที่สุดออกโดยอัตโนมัติ (Least Recently Used) การจำกัดขนาด” (maxsize) → เป็นหัวใจสำคัญของมัน

เช่น: มี maxsize = 3

```
cache = LRUDict(maxsize=3)
```

```
cache["A"] = "sound1"
```

```
cache["B"] = "sound2"
```

```
cache["C"] = "sound3"
```

```
# ตอนนี้เต็มแล้ว
```

```
cache["D"] = "sound4"
```

```
# → "A" จะถูกลบทิ้ง ( เพราะเก่าสุด)
```

💡 ทำไมต้องมี?

เพราะ:

- ไฟล์เสียงใหญ่
- RAM บอร์ดมีจำกัด
- 👉 เลย์ต้อง:
 - เก็บเฉพาะที่ “ใช้บ่อย”
 - ลบอันเก่าอัตโนมัติ

🎯 สรุป

👉 LRUDict =

“dict ที่ฉลาด รู้ว่าควรเก็บอะไร และควรลบอะไร”

- เก็บของที่ใช้บ่อย
- ลบของที่ไม่ได้ใช้

แบบธรรมดา	LRUDict
เก็บไม่จำกัด	จำกัดขนาด
ต้องลบเอง	ลบอัตโนมัติ
เสี่ยง memory เต็ม	ปลอดภัยกว่า

SoundGeneratorStreamer class

SoundGeneratorStreamer คือ “คลาสสำหรับ สร้างข้อมูลเสียง (audio data) แต่ไม่เล่นเสียงออกลำโพง”

🔧 ใช้งานทำอะไร?

ใช้เมื่อคุณต้องการ:

- 🌐 ส่งเสียงไป Web (WebUI / WebSocket)
- 📡 ส่ง audio ไป network / app อื่น
- 🎧 ประมวลผลเสียงเอง (ไม่ให้ออร์ดเล่น)

เช่น:

```
from arduino.app_bricks.sound_generator import SoundGeneratorStreamer

player = SoundGeneratorStreamer()

# สร้างเสียงโน้ต
audio, duration = player.play_tone("A4", 0.5)
```

👉 สิ่งที่ได้:

- audio = ข้อมูลเสียง (array)
- duration = ความยาวเสียง

📌 ยัง “ไม่มีเสียงออก” จนกว่าจะเอาไปใช้ต่อ

👉 **SoundGeneratorStreamer =**
เครื่องสร้างเสียง (data) สำหรับเอาไปใช้ต่อ
ไม่ได้เล่นเอง



Parameters มีอะไรบ้าง?

```
class SoundGeneratorStreamer(bpm: int, time_signature: tuple, octaves: int, wave_form: str, master_volume: float, sound_effects: list)
```

bpm (int)

☞ ความเร็วเพลง (จังหวะต่อนาที)

เช่น 120 = จังหวะมาตรฐาน

time_signature (tuple)

☞ จังหวะเพลง เช่น (4,4), (3,4)

บอกว่า 1 ห้องเพลงมีจังหวะยังงี้

octaves (int)

☞ จำนวนช่วงเสียง (ต่ำ → สูง) ที่ระบบสร้างได้
ยิ่งมาก → เล่นโน้ตได้กว้างขึ้น

wave_form (str)

☞ รูปแบบคลื่นเสียง

- sine = นุ่ม
- square = แนวเกม
- triangle = กลาง ๆ
- sawtooth = เสียงคม/แรง

master_volume (float)

☞ ความดังเสียง (0.0 – 1.0)

0 = เงียบ, 1 = ดังสุด

sound_effects (list) (optional)

☞ เอฟเฟกต์เสียง เช่น

- ADSR
- chorus
- overdrive

⚙️ เมธอดต่าง ๆ มีอะไรบ้าง?

🔊 1. set_wave_form(wave_form: str)

👉 เปลี่ยน “ลักษณะเสียงพื้นฐาน”

◆ Parameter wave_form

- "sine" → เสียงนุ่ม
- "square" → เสียงเหลี่ยม
- "triangle" → นุ่มแต่มีความคม
- "sawtooth" → เสียงแรง/คม

💡 ใช้เมื่อ:

อยากเปลี่ยน “โทนเสียง” ทั้งระบบ

🔊 2. set_master_volume(volume: float)

👉 ปรับความดังหลัก

◆ Parameter volume (0.0 – 1.0)

- 0 = เงียบ
- 1 = ดังสุด

💡 ใช้เมื่อ:

ควบคุมความดังรวมของทุกเสียง

🕒 3. set_bpm(bpm: int)

👉 ตั้งความเร็วเพลง

◆ Parameter bpm = beats per minute

เช่น:

- 60 = ช้า
- 120 = ปกติ
- 180 = เร็ว

💡 ใช้เมื่อ:

ใช้ play() ที่อิง duration แบบนับต

🔊 4. set_effects(effects: list)

👉 ใส่เอฟเฟกต์เสียง

◆ Parameter effects = list ของ effect เช่น

```
[SoundEffect.adsr(), SoundEffect.chorus()]
```

💡 ใช้เมื่อ:

อยากให้เสียงมี character เช่น เสียงหนา หรือ เสียงแตก

5. play(note, note_duration, volume)

👉 สร้างเสียง “โน้ตเดี่ยว”

◆ Parameters

note

เช่น "A4", "C#5", "REST" (เงียบ)

note_duration

- 1/4 = quarter note
- 1/8 = eighth note
- หรือ "Q", "H"

volume (optional)

- ถ้าไม่ใส่ → ใช้ master volume

Return

(audio_data) หรือ None

💡 ใช้เมื่อ:

เล่น melody ปกติ

6. play_tone(note, duration, volume)

👉 เหมือน play() แต่ใช้ “เวลาเป็นวินาที”

◆ Parameters

note เช่น "A4"

duration เช่น 0.5 (วินาที)

volume

Return

(audio_data)

💡 ใช้เมื่อ:

ต้องการควบคุมเวลาแบบ precise

7. play_chord(notes, note_duration, volume)

👉 เล่น “คอร์ด” (หลายโน้ตพร้อมกัน)

◆ Parameters

notes

เช่น ["C4", "E4", "G4"]

note_duration

เช่น 1/4

volume

Return

(audio_data ของคอร์ด)

💡 ใช้เมื่อ:

เล่น harmony / chord progression

8. play_polyphonic(notes, as_tone, volume)

 เล่น “หลายแท่งพร้อมกัน”

Parameters

notes

รูปแบบ:

```
[
    [ ("C4", 1/4), ("E4", 1/4) ], # melody
    [ ("C3", 1/2) ]                # bass
]
```

as_tone

- True → duration เป็น “วินาที”
- False → เป็น “โน้ต”

volume

Return

(audio_data, duration)

 ใช้เมื่อ:

ทำเพลงจริง (melody + bass)

9. play_abc(abc_string, volume)

 แปลง “ABC notation → เสียง”

Parameters

abc_string

เช่น:

C D E F G

volume

Return

(audio_data, duration)

 ใช้เมื่อ:

โหลดเพลงจาก text / standard notation

10. play_wav(wav_file)

 โหลดไฟล์เสียง .wav

Parameter

wav_file

เช่น "audio/sample.wav"

Return

(raw_pcm_data, duration)

จุดเด่น

- มี cache (ใช้ร่วมกับ LRUDict)
- โหลดเร็วขึ้นเมื่อใช้ซ้ำ

SoundGenerator class

👉 **SoundGenerator** คือ “คลาสสำหรับ สร้างและเล่นเสียงออกลำโพงจริง บนบอร์ด”

🧪 ตัวอย่างพื้นฐาน

```
from arduino.app_bricks.sound_generator import SoundGenerator
from arduino.app_utils import App

player = SoundGenerator()

def loop():
    player.play("A4", 1/4) # เล่นโน้ต A4

App.run(user_loop=loop)
```

👉 ผลลัพธ์: บอร์ดจะ “มีเสียงออกจริง” 🔊

Parameters มีอะไรบ้าง?

```
class SoundGenerator(output_device: Speaker, bpm: int, time_signature: tuple, octaves: int, wave_form: str, master_volume: float, sound_effects: list)
```

- **output_device (Speaker) (optional)**

👉 อุปกรณ์ที่ใช้เล่นเสียง (เช่น ลำโพง)
ถ้าไม่กำหนด ระบบจะใช้ลำโพงเริ่มต้น

- **bpm (int)**

👉 ความเร็วของเพลง (จังหวะต่อนาที)
ใช้คำนวณความยาวของโน้ต

- **time_signature (tuple)**

👉 รูปแบบจังหวะเพลง เช่น (4,4), (3,4)
บอกโครงสร้างของห้องเพลง

- **octaves (int)**

👉 จำนวนช่วงเสียง (octave) ที่สามารถสร้างได้
ยิ่งมาก → เล่นโน้ตได้กว้างขึ้น

- **wave_form (str)**

👉 รูปแบบคลื่นเสียง โดยรองรับ:

- "sine" (ค่าเริ่มต้น) = เสียงนุ่ม
- "square" = เสียงเหลี่ยม
- "triangle" = นุ่มกึ่งคม
- "sawtooth" = เสียงคม/แรง

- **master_volume (float)**

👉 ความดังเสียงหลัก (0.0 – 1.0)
0 = เงียบ, 1 = ดังสุด

- **sound_effects (list) (optional)**

👉 รายการเอฟเฟกต์เสียงที่ใช้
เช่น:

```
[SoundEffect.adsr()]
```


เมธอดต่าง ๆ มีอะไรบ้าง?

1. start()

เริ่มการทำงานของตัวสร้างเสียง (SoundGenerator) และลำโพงภายใน (ถ้าไม่ได้ใช้ ลำโพงภายนอก)

2. stop()

หยุดการเล่นเสียงทั้งหมด รวมถึงหยุด sequence ที่กำลังทำงาน และปิดลำโพงภายใน

3. set_master_volume(volume: float)

ตั้งค่าความดังเสียงหลัก

พารามิเตอร์

volume (float): ระดับความดัง (0.0 ถึง 1.0)

4. set_effects(effects: list)

ตั้งค่าเอฟเฟกต์เสียงที่จะนำมาใช้กับสัญญาณเสียง

พารามิเตอร์

effects (list): รายการเอฟเฟกต์ เช่น [SoundEffect.adsr()]

5. play_polyphonic(...)

เล่นโน้ตหลายชุดพร้อมกัน (multi-track / polyphony) โดยสามารถเล่นเพลงหลายแทร็กได้ โดยส่ง list ของ sequence ซึ่งแต่ละ sequence คือ list ของ (note, duration)

พารามิเตอร์

- notes: list ของ sequence (แต่ละตัวคือ list ของ (โน้ต, ระยะเวลา))
- as_tone: ถ้า True → duration เป็น “วินาที”
- volume: ความดัง (0.0–1.0) ถ้าไม่ใส่จะใช้ master volume
- block: ถ้า True → รอจนเล่นจบก่อนทำงานต่อ

🎵 6. **play_composition(composition, block)**

เล่นเพลงจาก object MusicComposition
ระบบจะตั้งค่าตาม composition แล้วเล่นผ่าน
play_step_sequence

พารามิเตอร์

- composition: เพลงที่ต้องการเล่น
- block: ถ้า True → รอจนเล่นจบ

🎹 7. **play_chord(notes, note_duration, volume, block)**

เล่นคอร์ด (หลายโน้ตพร้อมกัน)

พารามิเตอร์

- notes: list ของโน้ต เช่น ['A4', 'C#5', 'E5']
- note_duration: ระยะเวลา เช่น 1/4, 1/8 หรือ "Q"
- volume: ความดัง
- block: ถ้า True → รอจนเล่นจบ

🎵 8. **play(note, note_duration, volume, block)**

เล่นโน้ตเดี่ยว

พารามิเตอร์

- note: เช่น 'A4', 'C#5', 'REST'
- note_duration: ระยะเวลา (เช่น 1/4)
- volume: ความดัง
- block: ถ้า True → รอจนเล่นจบ

🕒 9. **play_tone(note, duration, volume, block)**

เล่นโน้ตโดยกำหนดเวลาเป็น “วินาที”
ต่างจาก play() ที่ใช้ duration แบบโน้ต

พารามิเตอร์

- note: โน้ต
- duration: เวลาเป็นวินาที
- volume: ความดัง
- block: ถ้า True → รอจนเล่นจบ

🎵 10. play_abc(abc_string, volume, block)

เล่นเพลงจากรูปแบบ ABC notation

รองรับมาตรฐาน ABC 2.1

พารามิเตอร์

- abc_string: ข้อความโน้ต
- volume: ความดัง
- block: ถ้า True → รอจนเล่นจบ

🎵 ABC Notation คืออะไร?

👉 ABC notation คือ

“รูปแบบการเขียนโน้ตดนตรีด้วยตัวอักษร (text)”

แทนที่จะเขียนเป็นโน้ตบนบรรทัด 5 เส้น

👉 เราเขียนเป็นตัวอักษรธรรมดาได้เลย

<https://abcnotation.com/examples>

X: 0
T: My Tune
M: 4/4
L: 1/4
K: C
C,D,E,F, | G,A,B,C | DEFG | ABcd | efga | bc'd'e' | f'g'a'b' |

K:C % key = C major
M:4/4 % time signature
L:1/4 % note length

My Tune





โครงสร้างพื้นฐาน

🎵 โน้ต

• C D E F G A B

👉 คือโน้ต



Octave

C = octave ปกติ

c = octave สูง

C, = octave ต่ำ



ระยะเวลา

C = ปกติ

C2 = ยาวขึ้น

C/2 = สั้นลง

ใน abc_string →

```
player.play_abc("""  
X:1  
T:Simple Song  
M:4/4  
K:C  
C D E F G  
""")
```

11. play_wav(wav_file, block)

เล่นไฟล์เสียง .wav

พารามิเตอร์

- wav_file: path ของไฟล์
- block: ถ้า True → รอจนเล่นจบ

12. play_step_sequence(...)

เล่น sequence แบบ step (เหมือน beat / loop)

ระบบจะจัดการ timing ให้เอง

พารามิเตอร์

- sequence: list ของ step (แต่ละ step เป็น list ของโน้ต)
- note_duration: ระยะเวลาของแต่ละ step
- bpm: ความเร็วเพลง
- loop: ถ้า True → เล่นวน
- on_step_callback: ฟังก์ชันที่เรียกทุก step
- on_complete_callback: ฟังก์ชันเมื่อเล่นจบ
- volume: ความดัง

Return

- ไม่มี (ทำงานแบบ background)

13.stop_sequence()

หยุด step sequence ที่กำลังเล่นอยู่ทันที

? 14.is_sequence_playing()

เช็คว่ามี sequence กำลังเล่นอยู่หรือไม่

Return

- True = กำลังเล่น
- False = ไม่ได้เล่น

```
# Simple melody with chords
sequence = [
    ["C4"], # Step 0: Single note
    ["E4", "G4"], # Step 1: Chord
    [], # Step 2: REST
    ["C5"], # Step 3: High note
]
sound_gen.play_step_sequence(sequence, note_duration=1 / 16, bpm=120)
```

MusicComposition class

👉 **MusicComposition** คือ

“คลาสสำหรับเก็บโครงสร้างเพลงทั้งหมดไว้ในรูปแบบข้อมูล (data)”

🧠 เข้าใจง่าย ๆ

👉 ถ้า **SoundGenerator** = คนเล่นเพลง 🎹

👉 **MusicComposition** = โน้ตเพลง / สกอร์เพลง 📄

📦 เก็บอะไรบ้าง?

- 🎵 โน้ต (note)
- 🕒 ความยาวโน้ต (duration)
- 🎵 BPM (ความเร็วเพลง)
- 🎛️ waveform
- 🔊 volume
- 🎛️ effects

ตัวอย่างใช้งาน

```
from arduino.app_bricks.sound_generator import MusicComposition, SoundGenerator, SoundEffect
from arduino.app_utils import App

comp = MusicComposition(
    composition=[
        [("C4", 1/4), ("E4", 1/4)], # track 1
        [("G4", 1/2)]               # track 2
    ],
    bpm=120,
    waveform="square",
    volume=0.8,
    effects=[SoundEffect.adsr()]
)

player = SoundGenerator()

def loop():
    player.play_composition(comp, block=True)

App.run(user_loop=loop)
```

คลาสนี้ใช้สำหรับ “เก็บข้อมูลเพลงทั้งหมดไว้ในที่เดียว”
เพื่อให้สามารถนำไปเล่น บันทึกลง หรือแชร์ได้ง่าย 🎵

Parameters มีอะไรบ้าง?

```
class MusicComposition(composition: list[list[tuple[str, float]]], bpm: int, waveform: str, volume: float, effects: list)
```

composition (list[list[tuple[str, float]]])

👉 ลำดับเพลงแบบหลายแทริก (polyphonic) ในรูปแบบ list ของ track แต่ละ track เป็น list ของ (โน้ต, ระยะเวลา) โดยระยะเวลาเป็นสัดส่วนของโน้ต เช่น

- 1/4 = quarter note
- 1/8 = eighth note

ตัวอย่าง:

```
[["C4", 0.25], ["E4", 0.25]], [{"G4", 0.5}]]
```

bpm (int)

👉 ความเร็วของเพลง (beats per minute)
ค่าเริ่มต้น: 120

waveform (str)

👉 รูปแบบคลื่นเสียง เช่น

- "sine"
- "square"
- "triangle"
- "sawtooth"

ค่าเริ่มต้น: "sine"

volume (float)

👉 ความดังเสียงหลัก (0.0 – 1.0)

ค่าเริ่มต้น: 0.8

effects (list)

👉 รายการเอฟเฟกต์เสียงที่ใช้

ค่าเริ่มต้น: [SoundEffect.adsr()]

SoundEffect class

SoundEffect คือคลาสที่ใช้สำหรับ “สร้างและจัดการเอฟเฟกต์เสียง”

🧠 เข้าใจง่าย ๆ

👉 ถ้า **SoundGenerator** = ตัวเล่นเสียง

👉 **SoundEffect** = ตัวแต่งเสียง

🎮 หน้าที่หลัก

✅ 1. เพิ่มเอฟเฟกต์ให้เสียง

เช่น:

- 🔥 overdrive → เสียงแตก
- 🎵 chorus → เสียงหนา
- 🎹 ADSR → คุม envelope
- 🌊 vibrato → เสียงสั่น
- ⚡ tremolo → เสียงกระพริบ

✅ 2. ใช้ร่วมกับ SoundGenerator

```
player = SoundGenerator(  
    sound_effects=[SoundEffect.adsr(), SoundEffect.chorus()]  
)
```

✅ 3. ปรับเสียงได้แบบ chain

👉 ใส่หลาย effect ต่อกันได้ เช่น

```
SoundEffect.adsr(),  
SoundEffect.overdrive(),  
SoundEffect.chorus()
```

⚙️ ทำงานยังไง (concept)

1. สร้างเสียงพื้นฐาน (waveform)
2. ส่งเข้า SoundEffect
3. เอฟเฟกต์แก้ไขสัญญาณเสียง
4. ส่งออกลำโพง

เมธอดต่าง ๆ มีอะไรบ้าง?

1. **overdrive(drive: float)**

👉 ใช้เพิ่มเอฟเฟกต์ “เสียงแตก” (overdrive) ให้กับสัญญาณเสียง

Parameters

- signal (np.ndarray)
👉 สัญญาณเสียงต้นฉบับ

- drive (float)

Returns

- (np.ndarray)
👉 สัญญาณเสียงที่ผ่านเอฟเฟกต์ overdrive แล้ว

2. **chorus(depth_ms, rate_hz: float, mix: float)**

👉 ใช้เพิ่มเอฟเฟกต์ “chorus” (เสียงซ้อน ทำให้เสียงหนา)

Parameters

- signal (np.ndarray)

👉 สัญญาณเสียงต้นฉบับ

- depth_ms (float)

👉 ความลึกของเอฟเฟกต์ (หน่วยมิลลิวินาที)
มีค่าระหว่าง 0 – 50 หน่วยเป็น ms
โดยมีค่าเริ่มต้นอยู่ที่ 10

- rate_hz (float)

👉 ความเร็วของการเปลี่ยนแปลงเสียง (Hz)

- mix (float)

👉 อัตราส่วนผสมระหว่างเสียงเดิม (dry) กับเสียงเอฟเฟกต์ (wet)

- 0.0 = เสียงเดิมล้วน
- 1.0 = เสียงเอฟเฟกต์ล้วน

Returns

- (np.ndarray)

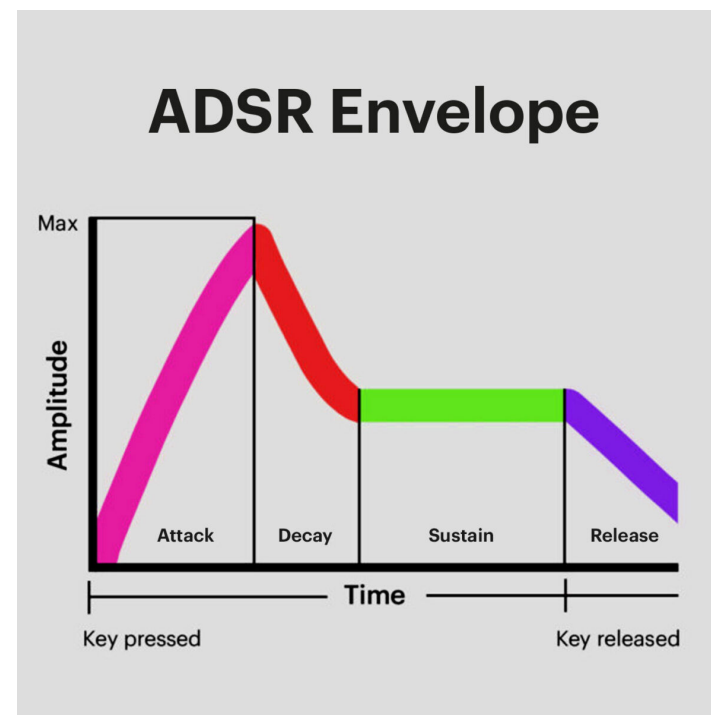
👉 สัญญาณเสียงที่ผ่านเอฟเฟกต์ chorus แล้ว

3. **adsr(attack, decay, sustain, release)**

👉 ใช้กำหนด “รูปทรงของเสียง” (envelope)

Parameters

- attack (float)
👉 เวลาที่เสียงค่อย ๆ ดังขึ้น (วินาที)
- decay (float)
👉 เวลาที่เสียงลดลงจากจุดสูงสุด
- sustain (float)
👉 ระดับเสียงที่คงอยู่
- release (float)
👉 เวลาที่เสียงค่อย ๆ เบาลงจนเงียบ



SoundEffectOverdrive class

👉 SoundEffectOverdrive คือ “คลาสสำหรับสร้างเอฟเฟกต์เสียงแตก (Overdrive)”

🎸 มีไว้ทำไม?

👉 ใช้เพื่อทำให้เสียง:

- หนาขึ้น
- ดุขึ้น
- มีความ “แตก” แบบกีตาร์ไฟฟ้า

🧠 เข้าใจง่าย ๆ

👉 เปรียบเทียบ:

- ไม่มี Overdrive → 🔊 เสียงใส เรียบ
- มี Overdrive → 🔥 เสียงแตก มีพลัง

⚙️ รูปแบบการใช้งาน

```
SoundEffectOverdrive(drive=100.0)
```

หรือใช้ผ่าน

```
SoundEffect.overdrive(drive=100.0)
```

📌 Parameter drive (float)

👉 ค่าความแรงของเสียงแตก

แนวโน้มค่า:

- ค่าน้อย → แตกนิด ๆ
- ค่ากลาง → แตกพอดี
- ค่าสูง → แตกหนัก 🔥

SoundEffectChorus class

👉 SoundEffectChorus คือ “คลาสสำหรับสร้างเอฟเฟกต์เสียงแบบ Chorus (เสียงซ้อน)”

🎧 **Chorus คืออะไร?**

👉 คือการทำให้เสียงเหมือน “มีหลายตัวเล่นพร้อมกัน”

- เสียงเดิม 1 เสียง 🎵
- ถูกหน่วงเวลา + ปรับ pitch เล็กน้อย
- กลายเป็นเสียง “หนา ฟุ้ง และกว้าง”

🧠 **เข้าใจง่าย ๆ**

👉 เปรียบเทียบ:

- ไม่มี Chorus → 🔊 เสียงเดียว แบบ ๆ
- มี Chorus → 🎵 เสียงหนา เหมือนมีหลายเครื่องเล่นพร้อมกัน

Parameters มีอะไรบ้าง?

```
class SoundEffectChorus(depth_ms: int, rate_hz: float, mix: float)
```

depth_ms (int)

👉 ความลึกของการหน่วงเสียง (มิลลิวินาที)

- มาก → เสียงฟุ้งมาก
- น้อย → เอฟเฟกต์เบา ๆ

rate_hz (float)

👉 ความเร็วในการสั่นของเอฟเฟกต์ (Hz)

- สูง → เสียงสั่นเร็ว
- ต่ำ → เสียงลอยนุ่ม ๆ

mix (float)

👉 สัดส่วนเสียง effect กับเสียงเดิม

- 0.0 → เสียงเดิมล้วน
- 1.0 → เอฟเฟกต์ล้วน
- 0.3–0.6 → กำลังดี 👍



ตัวอย่างใช้งาน

```
from arduino.app_bricks.sound_generator import SoundGenerator, SoundEffect

player = SoundGenerator(
    sound_effects=[
        SoundEffect.chorus(depth_ms=15, rate_hz=0.2, mix=0.4)
    ]
)
```

ใช้เมื่อไหร่?

-  ทำเสียงกีตาร์ให้ฟุ้ง
-  ทำเสียง synth ให้นุ่ม
-  ทำเสียง background / ambient
-  ทำเสียงร้องให้ดูเต็มขึ้น

SoundEffectADSR class

👉 SoundEffectADSR คือ คลาสที่ใช้กำหนด “Envelope ของเสียง”

หรือก็คือ

👉 รูปแบบการ ขึ้น-ค้าง-ลง-ดับ ของเสียง

🧠 ADSR คืออะไร? ADSR = 4 ช่วงของเสียง

1. Attack ⬆️

👉 เสียงเริ่มจาก 0 → ดังสุด ใช้เวลานานแค่ไหน

2. Decay ⬇️

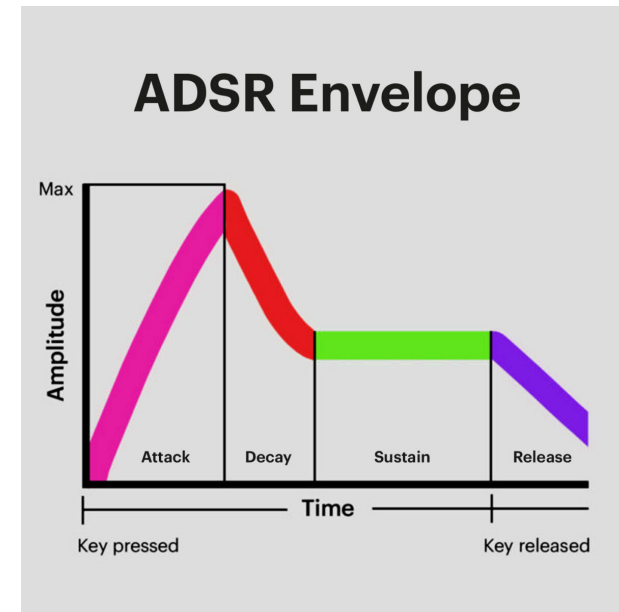
👉 หลังจากดังสุด → ลดลงมาถึงระดับคงที่

3. Sustain —

👉 ระดับเสียงที่ค้างไว้ขณะกดโน้ต

4. Release ⬇️

👉 เวลาที่เสียงค่อย ๆ หายไปหลังปล่อยโน้ต



รูปแบบการใช้งาน

```
SoundEffectADSR(attack, decay, sustain, release)
```

⚙️ Parameters มีอะไรบ้าง?

```
class SoundEffectADSR(attack: float, decay: float, sustain: float, release: float)
```

- **attack (float)**

👉 เวลาเสียงขึ้น (วินาที)

- **decay (float)**

👉 เวลาลดจาก peak ลงมา

- **sustain (float)**

👉 ระดับเสียงค้าง (0.0 – 1.0)

- **release (float)**

👉 เวลาที่เสียงค่อย ๆ หาย

🎵 ผลลัพธ์ที่ได้

- attack ต่ำ → เสียงมาไว ⚡
- attack สูง → เสียงค่อย ๆ มา 🌊
- release สั้น → ตัดเร็ว
- release ยาว → เสียงลอย 🎵

ตัวอย่างใช้งาน →

```
player = SoundGenerator(  
    sound_effects=[  
        SoundEffect.adsr(  
            attack=0.01,  
            decay=0.2,  
            sustain=0.7,  
            release=0.3  
        )  
    ]  
)
```

⚙️ เมธอดต่าง ๆ มีอะไรบ้าง?

`apply(signal: np.ndarray)`

👉 ใช้สำหรับ “นำเอฟเฟกต์ ADSR ไปปรับกับสัญญาณเสียง”

`apply(...)` 👉 คือฟังก์ชันที่ “เอา ADSR ไปใส่ในเสียงจริง”



Parameters

`signal: np.ndarray (float32)`

👉 สัญญาณเสียงต้นฉบับ
(เป็นข้อมูลเสียงในรูปแบบ array ของตัวเลข)



ทำงานยังไง?

👉 ขั้นตอน:

- 1.รับเสียงต้นฉบับ (signal)
- 2.เอาค่า ADSR (attack, decay, sustain, release)
- 3.ปรับความดังของเสียงตามช่วงเวลา
- 4.ส่งเสียงใหม่กลับมา



เข้าใจง่าย

👉 ก่อน apply:

- เสียง = ดังเท่ากันตลอด

👉 หลัง apply:

- เสียง = ค่อย ๆ ดัง → ลด → ค้าง → หาย

SoundEffectTremolo class

SoundEffectTremolo คือ คลาสสำหรับสร้างเอฟเฟกต์ “Tremolo”

🎧 **Tremolo คืออะไร?**

- 👉 Tremolo = การ “สั่นของความดังเสียง”
- เสียงจะดัง-เบา-ดัง-เบา สลับกันเร็ว ๆ

🧠 **เข้าใจง่าย**

👉 เปรียบเทียบ:

- ไม่มี Tremolo → 📢 เสียงนิ่ง
- มี Tremolo → 🌊 เสียงกระพริบ / เต้น

⚙️ **รูปแบบการใช้งาน**

```
SoundEffectTremolo(depth, rate)
```

📌 **Parameters**

depth (float)

- 👉 ความแรงของเอฟเฟกต์ (0.0 – 1.0)
- 0 = ไม่มีผล
 - 1 = สั่นเต็มที่

rate (float)

- 👉 ความเร็วในการสั่น (จำนวนรอบต่อ block)
- สูง → สั่นเร็ว ⚡
 - ต่ำ → สั่นช้า 🌊

ตัวอย่างใช้งาน

```
player = SoundGenerator(  
    sound_effects=[  
        SoundEffect.tremolo(depth=0.7, rate=5.0)  
    ]  
)
```

👉 เสียงจะ “ขึ้น ๆ ลง ๆ” แบบเป็นจังหวะ

💡 **Insight**

👉 Tremolo = “Amplitude modulation” (ปรับความดังขึ้นลงตามเวลา)

⚙️ เมธอดต่าง ๆ มีอะไรบ้าง?

apply(signal: np.ndarray)

👉 ใช้สำหรับ

“นำเอฟเฟกต์ Tremolo ไปปรับกับสัญญาณเสียง”

apply(...) 👉 ฟังก์ชันที่เอาเอฟเฟกต์ Tremolo ไปใส่ในเสียงจริง

📁 **Parameters**

signal (np.ndarray)

👉 ข้อมูลเสียงที่รับเข้ามา (audio block) เป็น array ของตัวเลขที่แทนสัญญาณเสียง

⚙️ **ทำงานยังไง?**

👉 ขั้นตอน:

- 1.รับเสียงต้นฉบับ (signal)
- 2.สร้างรูปแบบการ “สั่นของความดัง” (ตามค่า depth และ rate)
- 3.ปรับความดังของเสียงให้ขึ้น-ลงตามจังหวะ
- 4.ส่งเสียงใหม่กลับออกไป

🧠 **เข้าใจง่าย**

👉 ก่อน apply:

- 🎧 เสียงดังเท่ากันตลอด

👉 หลัง apply:

- 🌊 เสียงดัง-เบา-ดัง-เบา เป็นจังหวะ

🎯 **สรุปสั้น**

👉 apply() ของ Tremolo คือ ฟังก์ชันที่ “ทำให้เสียงกระพริบ (ดัง-เบา)” ตามจังหวะที่กำหนด 🌊

SoundEffectVibrato class

👉 SoundEffectVibrato คือ คลาสสำหรับสร้างเอฟเฟกต์ “Vibrato”

🎧 **Vibrato คืออะไร?**

👉 Vibrato = การ “สั่นของความถี่เสียง (pitch)”
เสียงจะสูง-ต่ำ-สูง-ต่ำ สลับกันเร็ว ๆ

🧠 **เข้าใจง่าย**

👉 เปรียบเทียบ:

- ไม่มี Vibrato → 🗣️ เสียงนิ่ง
- มี Vibrato → 🎵 เสียงสั่นแบบนักร้อง

⚙️ **รูปแบบการใช้งาน**

`SoundEffectVibrato(depth, rate)`

depth (float)

👉 ความแรงของการสั่น
(pitch deviation)

- 0 = ไม่มีเอฟเฟกต์
- 0.5 = สั่นแรงมาก

rate (float)

👉 ความเร็วในการสั่น (จำนวนรอบต่อ block)

- สูง → สั่นเร็ว ⚡
- ต่ำ → สั่นช้า 🌊

🧪 ตัวอย่างใช้งาน

```
from arduino.app_bricks.sound_generator import SoundGenerator, SoundEffect

player = SoundGenerator(
    sound_effects=[
        SoundEffect.vibrato(depth=0.2, rate=5.0)
    ]
)
```

⚠️ **อย่าสับสน**

- Vibrato → 🎵 สั่น “ความถี่เสียง (pitch)”
- Tremolo → 🗣️ สั่น “ความดัง (volume)”

SoundEffectBitcrusher class

👉 SoundEffectBitcrusher คือ คลาสสำหรับทำให้เสียง “แตกแบบดิจิตอล”

🎧 **Bitcrusher คืออะไร?**

👉 เป็นเอฟเฟกต์ที่ “ลดคุณภาพเสียงโดยตั้งใจ”

- ลดความละเอียดของเสียง (bit depth)
- ลดจำนวน sample (sampling rate)

👉 ผลลัพธ์:

- เสียงหยาบ
- เสียงแตกแบบ digital
- ฟील retro / 8-bit 🎮

🧠 **เข้าใจง่าย**

👉 เปรียบเทียบ:

- เสียงปกติ → 🎵 ไส้ คม
- Bitcrusher → 🎮 แตก หยาบ แบบเกมเก่า

⚙️ รูปแบบการใช้งาน

```
SoundEffectBitcrusher(bits, reduction)
```

1. bits (int)

👉 ความละเอียดของเสียง (bit depth)

- 16 = เสียงปกติ (คุณภาพสูง)
- 8 = เริ่มหยาบ
- 4 = หยาบมาก
- 1 = แตกสุด 🔥

👉 ยิ่งน้อย → ยิ่งแตก

2. reduction (int)

👉 การลด sampling rate (downsampling)

- 1 = ไม่ลด
- 2 = ลดครึ่งหนึ่ง
- 4 = ลดหนัก
- 8+ = หยาบมาก

👉 ยิ่งมาก → เสียงยิ่ง “กระตุก”



ตัวอย่างใช้งาน

```
from arduino.app_bricks.sound_generator import SoundGenerator, SoundEffect

player = SoundGenerator(
    sound_effects=[
        SoundEffect.bitcrusher(bits=4, reduction=4)
    ]
)
```

🎯 ใช้เมื่อไหร่?

- 🎮 ทำเสียงเกม 8-bit / retro
- 🎵 ทำ lo-fi music
- 🤖 ทำเสียง glitch / AI effect
- 🎧 sound design แนว experimental



Insight

👉 Bitcrusher = “ลดข้อมูลเสียง”
ทำให้เกิด artifact → กลายเป็นสไตล์

SoundEffectOctaver class

👉 SoundEffectOctaver คือ คลาสสำหรับ “เพิ่มเสียงที่สูง/ต่ำกว่าต้นฉบับ 1 octave”

🎧 **Octaver คืออะไร?**

👉 Octave = ระดับเสียงที่ห่างกัน 8 โน้ต

เช่น:

- C4 → C5 (สูงขึ้น 1 octave)
- C4 → C3 (ต่ำลง 1 octave)

🧠 **เข้าใจง่าย**

👉 เอฟเฟกต์นี้จะ:

- เล่นเสียงเดิม 🎵
- เพิ่มเสียงสูงขึ้น ⬆️
- เพิ่มเสียงต่ำลง ⬇️

👉 กลายเป็นเสียง “หนาและเต็มขึ้น”

⚙️ **รูปแบบการใช้งาน**

```
SoundEffectOctaver(oct_up, oct_down)
```

oct_up (bool)

👉 เพิ่มเสียง “สูงขึ้น 1 octave”

- True = เปิด
- False = ปิด

oct_down (bool)

👉 เพิ่มเสียง “ต่ำลง 1 octave”

- True = เปิด
- False = ปิด

🧪 **ตัวอย่างใช้งาน**

```
from arduino.app_bricks.sound_generator import SoundGenerator, SoundEffect

player = SoundGenerator(
    sound_effects=[
        SoundEffect.octaver(oct_up=True, oct_down=True)
    ]
)
```

ตัวอย่างการตั้งค่า

 เพิ่มแต่เสียงต่ำ

```
SoundEffect.octaver(oct_up=False, oct_down=True)
```

 เพิ่มแต่เสียงสูง

```
SoundEffect.octaver(oct_up=True, oct_down=False)
```

 เพิ่มทั้งคู่

```
SoundEffect.octaver(oct_up=True, oct_down=True)
```

เมธอดต่าง ๆ มีอะไรบ้าง?

apply(signal)

👉 ใช้สำหรับ:

“เอาเอฟเฟกต์ Octaver ไปใส่ในเสียงจริง”

Parameter

signal (np.ndarray)

👉 ข้อมูลเสียง (ค่าอยู่ในช่วง -1 ถึง 1)

ทำงาน:

- 1.รับเสียงต้นฉบับ
- 2.สร้างเสียง octave สูง/ต่ำ
- 3.ผสมเข้ากับเสียงเดิม
- 4.ส่งออกเสียงใหม่

ใช้เมื่อไหร่?

-  ทำเสียงกีตาร์ให้หนา
-  ทำ bass ให้ลึกขึ้น
-  ทำ sound effect เกม
-  ทำ synth ให้เต็ม

Insight

👉 Octaver = “เพิ่ม layer ของเสียง”
ไม่ใช่แค่เปลี่ยนเสียง แต่ “เพิ่มอีกเสียง”

สรุปสั้น

👉 SoundEffectOctaver คือ
คลาสสำหรับ “เพิ่มเสียง octave สูง/ต่ำ”
เพื่อทำให้เสียง “หนาและเต็มขึ้น” 🎵

WaveSamplesBuilder class

👉 WaveSamplesBuilder คือ คลาสสำหรับ “สร้างคลื่นเสียง (audio waveform) แบบดิบ ๆ”

🧠 เข้าใจง่าย

👉 ถ้าเทียบทั้งระบบ:

- 🎹 SoundGenerator = เล่นเพลง
- 🎛️ SoundEffect = แต่งเสียง
- 🎧 **WaveSamplesBuilder = สร้างเสียงดิบ**

🎧 หน้าที่หลัก

👉 สร้าง “คลื่นเสียง” เช่น:

- sine → เสียงนุ่ม
- square → เสียงเกม 8-bit
- triangle → เสียงใส
- sawtooth → เสียง synth
- white_noise → เสียงซ่า

⚙️ รูปแบบการใช้งาน

```
WaveSamplesBuilder(wave_form, sample_rate)
```

wave_form (str)

👉 รูปแบบคลื่นเสียง

เช่น "sine", "square", "triangle", "sawtooth", "white_noise"

sample_rate (int)

👉 ความละเอียดของเสียง (Hz)

เช่น:

- 44100 = คุณภาพ CD 🎵
- 22050 = กลาง ๆ
- ต่ำ = เสียงหยาบ

⚙️ เมธอดต่าง ๆ มีอะไรบ้าง?

◆ generate_block(...)

```
generate_block(freq, block_dur, master_volume)
```

📁 Parameters

freq (float)

👉 ความถี่เสียง (Hz)

เช่น:

- 440 = A4 🎵

block_dur (float)

👉 ระยะเวลาเสียง (วินาที)

master_volume (float)

👉 ความดัง (0.0 – 1.0)

📁 Returns

numpy.ndarray 👉 ข้อมูลเสียง (array) ที่เอาไปเล่นต่อได้

🧪 ตัวอย่างใช้งาน

```
from arduino.app_bricks.sound_generator import WaveSamplesBuilder

builder = WaveSamplesBuilder(wave_form="sine", sample_rate=44100)

audio = builder.generate_block(
    freq=440,
    block_dur=1.0,
    master_volume=0.8
)
```

👉 ได้เสียง:

- โน้ต A (440 Hz)
- ยาว 1 วินาที

⚙️ ทำงานยังไง?

1. เลือก waveform
2. คำนวณค่าคลื่น (sin / square ฯลฯ)
3. สร้าง array ของ sample
4. ส่งออกเป็นข้อมูลเสียง

🎯 ใช้เมื่อไหร่?

- 🎵 สร้างเสียงเองแบบ low-level
- 🧠 ทำ AI audio / DSP
- 🎮 ทำ sound engine
- 🔬 ทดลองเรื่อง signal processing

ABCNotationLoader class

👉 ABCNotationLoader คือ ตัวแปลงโน้ตเพลงแบบตัวอักษร (ABC notation) ➡ ให้กลายเป็นข้อมูลที่เครื่องเอาไป “เล่นเสียง” ได้

เมธอด

◆ parse_abc_notation(...)

parse_abc_notation(abc_string, default_octave)

abc_string (str)

👉 ข้อความเพลงแบบ ABC notation

default_octave (int)

👉 กำหนด octave เริ่มต้น

เช่น:

- C = C4 (default)
- หรือจะให้เป็น C3 ก็ได้

📁 Returns

(metadata, notes)

◆ **metadata (dict)**

👉 ข้อมูลเพลง เช่น:

- title (T:)
- key (K:)
- tempo (Q:)
- meter (M:)

◆ **notes (list)**

👉 รายการโน้ต เช่น:

```
[  
    ("C4", 0.25),  
    ("D4", 0.25),  
    ("E4", 0.5)  
]
```

🖋 ตัวอย่างใช้งาน

```
from arduino.app_bricks.sound_generator import ABCNotationLoader  
  
abc = """  
X:1  
T:Simple Song  
M:4/4  
K:C  
C D E F G  
"""  
  
loader = ABCNotationLoader()  
  
metadata, notes = loader.parse_abc_notation(abc, default_octave=4)  
  
print(notes)
```

🔥 ใช้ร่วมกับ SoundGenerator

```
from arduino.app_bricks.sound_generator import SoundGenerator

player = SoundGenerator()

for note, duration in notes:
    player.play(note, duration)
```

👉 เท่ากับ “เล่นเพลงจากตัวหนังสือ”

🎯 จุดเด่นของคลาสนี้

- ✅ เขียนเพลงแบบ text ได้
- ✅ รองรับมาตรฐาน ABC 2.1
- ✅ ใช้กับ AI / generative music ได้
- ✅ เอาไปต่อกับ Web / Bot ได้ง่าย

🎯 สรุปสั้น

👉 ABCNotationLoader คือ

ตัวแปลง “โน้ตเพลงแบบตัวอักษร (ABC)” →
เป็นข้อมูลโน้ตที่เล่นได้

💡 Insight สำคัญ

👉 ทำให้คุณสามารถ:

- พิมพ์เพลง → เล่นได้เลย 🎵
- โหลดเพลงจาก text → เล่นได้
- ให้ AI generate เพลง → เล่นได้ทันที 🤖